NTNU

Medical Image Segmentation of Head and Neck Tumors for MRI-Guided Radiation Therapy Planning

Marcos Barreiro
1st May 2025
Department of Computer Science, NTNU

Correspondence: marcosba@ntnu.no

Abstract

Background: Accurate segmentation of tumors in 3D MRI is essential for radiation therapy planning in head and neck cancer (HNC) patients. Manual annotation is time-consuming and prone to variability, prompting interest in deep learning-based automation.
Methods: We developed a deep learning pipeline using MONAI and PyTorch, evaluating several segmentation architectures and loss functions through a structured preselection phase. Our final model, DynUNet without deep supervision trained with BCE + Dice loss, was refined through a two-phase training process. Validation logic was corrected mid-training after identifying a misalignment in Dice computation.
Results: The model achieved an average test-time Dice score of 0.7024. We visualized predictions using 3D Slicer and exported best, mid-range, and worst-performing cases for qualitative assessment. Total compute time was estimated at 22.4 hours, consuming 10.09 kWh—equivalent to driving a Tesla Model Y for 63 km.
Conclusions: This project demonstrated how a modular training system, combined with rigorous validation and domain-specific preprocessing, can deliver clinically meaningful segmentation performance. The work strengthened my understanding of medical imaging, robust training, and project structuring in real-world ML workflows.

Keywords

Keywords: Medical image segmentation, DynUNet, MONAI, MRI, Radiation therapy, Dice score, Deep learning, Head and neck cancer

Motivation

As a student exploring the intersection of machine learning and the life sciences, I have developed a growing interest in the field of bioinformatics. My prior academic work includes a project where I designed a Random Forest-based classifier to predict the pathogenicity of somatic cancer mutations. Such study reinforced for me how data-driven approaches, when combined with domain knowledge, can uncover clinically meaningful patterns.

This medical segmentation project presented an opportunity to continue exploring this intersection, but in a spatial and imaging context. Unlike the tabular genomics data I had worked with previously, MRI volumes pose unique challenges—including data sparsity, anisotropy, and complex spatial structures. Engaging with these challenges has allowed me to expand both my technical toolkit and my understanding of how AI can support decision-making in medical diagnostics and treatment planning.

Moreover, working with real 3D clinical MRI data in a segmentation setting simulates a real-world application where AI can provide tangible value, potentially improving patient outcomes. This project has further confirmed my interest in applying deep learning to biomedical problems and has motivated me to consider bioinformatics and medical image computing as serious directions for future study and research.

Background

Radiation therapy (RT) is a cornerstone in the treatment of head and neck cancer (HNC), a category of tumors that arise in anatomically dense and functionally critical regions such as the pharynx, larynx, and oral cavity. Due to the proximity of vital organs like the spinal cord, salivary glands, and optic nerves, accurate delineation of tumor boundaries is crucial for effective treatment and to minimize collateral damage to healthy tissues [1]. Magnetic Resonance Imaging (MRI) offers superior soft tissue contrast compared to Computed Tomography (CT), and is increasingly used in RT planning to improve tumor visibility and anatomical localization [2].

However, one of the key bottlenecks in the MRI-based radiotherapy workflow is the manual segmentation of tumors—a process that is time-consuming, requires clinical expertise, and is prone to inter-observer variability [3]. This has motivated extensive research into automated image segmentation methods, particularly those based on deep learning (DL), which have shown remarkable performance across various medical imaging tasks [4].

Image segmentation, in the context of medical imaging, refers to the process of assigning a semantic label (e.g., tumor, organ, background) to each voxel or pixel in an image volume. In clinical practice, segmentation is critical not only for diagnostic purposes but also for treatment planning and monitoring disease progression. Medical segmentation models must address challenges such as data sparsity, class imbalance, and high inter-patient variability.

One of the most successful deep learning architectures for segmentation is the U-Net, first introduced for biomedical image segmentation [5]. The U-Net follows an encoder–decoder structure, where the encoder captures contextual features through convolution and pooling operations, while the decoder progressively restores spatial resolution via upsampling and skip connections. These skip connections directly link feature maps from the encoder to the decoder, helping preserve fine-grained spatial information that is often lost during downsampling. This structure enables the model to learn both global context and precise boundary localization, which is crucial for tasks like tumor segmentation.

For volumetric data, such as 3D MRI scans, a natural extension is the 3D U-Net [6], which replaces 2D operations with 3D convolutions and pooling layers to capture volumetric context. Despite its success, the 3D U-Net can struggle with diverse spatial resolutions and large anatomical variations, especially when the dataset contains highly anisotropic voxel spacings or variable field-of-view sizes.

To address these limitations, more adaptive architectures have been proposed. In this project, we employed the DynUNet model, introduced in the nnU-Net framework [7]. DynUNet is a dynamically configurable architecture that adjusts its depth, kernel sizes, and strides based on the input image resolution. It supports deep supervision, where intermediate layers also contribute to the final loss, helping to regularize the network and accelerate convergence. Additionally, DynUNet introduces residual units and flexible convolutional block configurations, enabling better feature reuse and gradient flow in deep architectures.

To facilitate implementation and reproducibility, we used MONAI (Medical Open Network for AI) [8], a PyTorch-based framework specifically designed for healthcare imaging. MONAI offers out-of-the-box support for medical formats (NIfTI, DICOM), patch-based training, complex 3D transformations, and evaluation metrics like the Dice coefficient. Its modular design allowed us to

rapidly prototype pipelines, compare architectures, and conduct experiments under consistent settings.

By combining these theoretical and practical tools, this project aimed to implement an efficient and accurate segmentation pipeline for HNC tumor localization in pre-treatment MRI scans, contributing to the automation of MRI-guided radiotherapy planning.

Approach

Our solution to the HNTS-MRG preRT segmentation task was designed to be modular, iterative, and data-informed. The pipeline was orchestrated through a Jupyter notebook controlling execution, supported by modular Python scripts encapsulating data I/O, training logic, loss computation, evaluation, and configuration.

Code Architecture

The codebase was divided into distinct modules:

- `data_analysis.ipynb`: Performed EDA to inspect voxel intensity distributions, tumor class sparsity, and volume shapes. This guided our data normalization and augmentation strategy.
- `dataset.py`: Defined a MONAI-based `CacheDataset` for efficient patch-based loading of 3D NIfTI MRI–mask pairs.
- `transforms.py`: Composed MONAI transforms for spatial standardization and data augmentation in both strong (training) and conservative (validation/test) modes.
- `model.py`: Provided a unified interface to construct and configure candidate architectures.
- `loss.py`: Included implementations of loss functions suitable for segmentation under class imbalance.
- `metrics.py`: Integrated MONAI's `compute_meandice` to evaluate overlap metrics per batch and per epoch.
- `train.py`: Contained the full training loop, validation hooks, early stopping, and logging logic.
- `utils.py`: Provided learning rate scheduling, visualization tools, and reproducibility helpers.
- `config.py`: Defined centralized training parameters, device configuration, and I/O paths.

Stage 0: Data Analysis and Transform Design

The pipeline began with exploratory analysis using `data_analysis.py`, which revealed extreme voxel-level class imbalance, heterogeneous tumor sizes, and intensity variation. These findings motivated our selection of class-sensitive loss functions, intensity normalization strategies, and a robust patch sampling scheme. In particular, we adopted intensity windowing and contrast-preserving normalization, as well as foreground cropping and class-balanced random sampling within patches.

Stage 1: Preliminary Evaluation Loop

Before committing to full-scale training, we ran a systematic preselection loop across multiple combinations of architectures and loss functions using a reduced dataset. Each candidate was trained for a limited number of epochs, and model performance was tracked using training loss and validation metrics. This process helped identify stable configurations for full training.

Stage 2: Learning Rate Scheduling and Validation Setup

We applied a dynamic learning rate schedule combining linear warm-up with cosine annealing [10], and configured early stopping to monitor validation performance. To ensure correctness, validation was integrated directly into the training loop, with Dice metrics computed and logged throughout training. Inference was performed using MONAI's `sliding_window_inference()` with adaptive ROI size and overlap parameters.

Stage 3: Full Training

Full training was performed using mixed-precision computation and patch-based sampling. Training ran for up to a maximum number of epochs, with validation checkpoints and early stopping based on Dice score progression. The best-performing model (based on validation metrics) was checkpointed for use in test-time evaluation.

Stage 4: Inference and Visualization Pipeline

After training, the best model was applied to the test set using a custom inference script. Predictions were post-processed and stored in NIfTI format for both quantitative evaluation and qualitative visualization. A utility was developed to automate export and overlay generation in tools such as 3D

Slicer, enabling visual inspection of segmentation results in both 2D and 3D views.

Data Analysis

To ensure our preprocessing and training pipeline was adapted to the characteristics of the dataset, we performed an exploratory analysis of the 130 preRT MRI volumes. This phase focused on quantifying tumor volume distribution, class imbalance, spatial variability, and intensity statistics. These insights were instrumental in shaping our data transformations and augmentation strategy.

Tumor Volume and Spatial Variability

All subjects presented annotated tumor regions, yet these varied substantially in size—from 1,902 mm^3 to 126,771 mm^3, with a mean volume of 23,347 mm^3 and a median of 17,890 mm^3. The dataset originally included two separate tumor classes; however, for the purpose of this project, the labels were binarized into a single "tumor" class, combining both annotations into a unified mask. This simplification reduced label noise and focused the learning objective on tumor versus background segmentation.

Additionally, bounding box analysis revealed strong anatomical variability. As shown in Figure 1, the extent of tumors in the axial (X, Y) dimensions was broad, while the depth (Z) was typically more constrained—reflecting the anisotropic resolution of the MR images.

These findings guided several decisions: to ensure tumor coverage during training, we adopted a patch size of 128^3, and used class-sensitive loss functions such as Dice and BCE+Dice. The observed spatial heterogeneity also led us to use spatial padding and patch sampling techniques that adapt to tumor size and location.

Preprocessing and Augmentation Strategy

The data analysis also revealed high inter-patient variation in voxel intensities within tumor regions. To mitigate this, we applied intensity normalization to a fixed window (0–300 HU), scaling to the [0,1] range using MONAI's `ScaleIntensityRanged` transform. Histogram equalization was deliberately avoided to preserve biologically relevant contrast variations.

For augmentation, we designed a dual-mode pipeline using MONAI's transforms: a base version for validation and testing, and a stronger variant for training with probabilistic application of bias field, Gaussian noise, contrast shifts, affine deformations, and elastic warping. The choice and strength of augmentations were informed by the need to maintain tumor shape fidelity while introducing robustness to anatomical variance.

Preprocessing steps included:

- `Spacingd` and `Orientationd` to standardize voxel spacing and axes.
- Foreground cropping with `CropForegroundd` to reduce background redundancy.
- `RandCropByPosNegLabeld` to balance foreground and background sampling within each patch.
- `SpatialPadd` to ensure patches were aligned and uniformly sized.

The validation and test transforms retained all geometric and intensity corrections but excluded random augmentations to ensure consistent and unbiased evaluation.

Additional plots illustrating tumor voxel ratios, bounding box volumes, and intensity distribution statistics are available in Appendix A.

Models

Architectures

We implemented and evaluated three encoder–decoder architectures in MONAI, all adapted to 3D volumetric data:

- **UNet** [6]: A baseline 3D convolutional architecture with skip connections.
- **DynUNet** [7]: A spacing-aware model that dynamically adjusts depth and kernel size based on input resolution. A variant with deep supervision was also considered but was excluded from the preselection experiments due to significantly increased computational overhead and prolonged training times.
- **Attention UNet** [13]: Extends UNet with spatial attention mechanisms to enhance learning on relevant features, potentially aiding small tumor detection.

All models used a consistent input shape of 128^3 and had output channels equal to one (binary segmentation).

Loss Functions

Three loss functions were explored:

- **Dice loss**: Optimizes the overlap between predicted and ground truth masks.

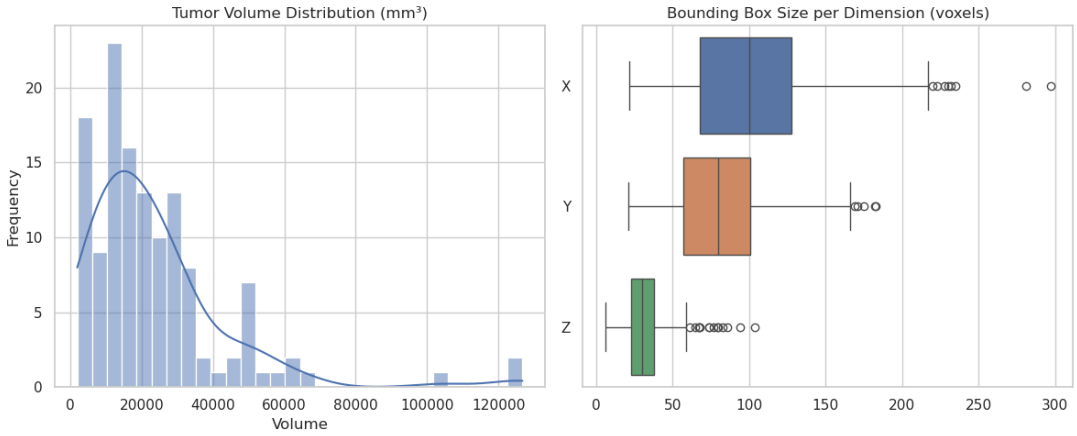

Figure 1: Tumor volume distribution (left) and bounding box dimension variability (right).

- **Tversky loss** [12]: A generalization of Dice, balancing false positives and negatives via tunable hyperparameters ($\alpha = 0.7$, $\beta = 0.3$).
- **BCE + Dice**: A weighted combination of binary cross-entropy (30%) and Dice loss (70%) for both spatial and voxel-level accuracy.

Training and Validation Procedure

Model training was handled by a custom `train_and_validate()` function, which supported deep supervision, mixed precision training, learning rate scheduling, sliding-window inference, and early stopping.

At each epoch:

- The model was trained using `autocast()` and `GradScaler()` to optimize memory usage.
- Deep supervision outputs (if any) were interpolated and combined using the same loss function.
- Validation was performed every few epochs using MONAI's `sliding_window_inference()` with adaptive ROI size and overlap.
- Soft Dice scores were averaged over batches using MONAI's `DiceMetric`.

To prevent premature convergence, we used cosine annealing with warm-up to adapt the learning rate over time. Early stopping was triggered after 20 validation steps without Dice improvement, provided a minimum of 100 epochs had been completed.

Model Preselection Results

Each model–loss combination was trained on a reduced dataset (2–3 patients) for up to 100 epochs with a small patch size (96^3). Due to resource constraints, DynUNet with deep supervision was excluded from this batch evaluation, as its multi-output structure considerably increased memory usage and training time.

The best validation Dice scores obtained from the preselection phase are summarized in Table 1 (see Appendix B for more information about the models' training performance).

Table 1

Model	Loss	Best Dice
UNet	BCE + Dice	0.0750
UNet	Dice	0.0299
UNet	Tversky	0.0734
DynUNet (no DS)	BCE + Dice	**0.3084**
DynUNet (no DS)	Dice	0.2303
DynUNet (no DS)	Tversky	0.1870
Attention UNet	BCE + Dice	0.2347
Attention UNet	Dice	0.0029
Attention UNet	Tversky	0.0000

DynUNet without deep supervision, trained with the BCE + Dice loss, consistently outperformed other configurations and was selected for full training.

Results

First Training Round

After selecting DynUNet (no deep supervision) with BCE + Dice loss, we initiated full training for 500 epochs using the `train_and_validate()` function with early stopping enabled (patience = 20, minimum 100 epochs). The model showed consistent loss reduction across training epochs (Figure 2, left). However, the validation Dice remained anomalously low, plateauing around 0.07 (Figure 2, right).

This unexpected discrepancy raised concerns regarding the Dice computation during validation. To investigate further, we conducted a manual sanity check using direct inference on selected validation samples followed by thresholding and Dice calculation.

Sanity Check and Diagnosis

Despite low Dice values during training, qualitative inspection of the output masks revealed that predictions were structurally coherent and closely matched the ground truth (Figure 3). Quantitative Dice scores computed manually on 25 validation samples ranged from 0.45 to 0.88, with many above 0.80. This confirmed that the training itself was effective, and the issue lay in the way Dice was computed during validation: the validation Dice was being calculated on raw (unthresholded) sigmoid outputs instead of binarized predictions.

Second Training Round

After identifying the issue, we retained the model weights and resumed training with corrected validation logic. The learning rate was reduced to 5×10^{-5}, and training continued for an additional 500 epochs. This time, validation Dice improved steadily, peaking near 0.70 before early stopping was triggered around epoch 450 (Figure 4).

Final Test Evaluation and 3D Slicer Export

The final model was evaluated on the held-out test set using MONAI's `compute_meandice()` function applied to binarized outputs. We obtained an average Dice score of:

0.7024

To enable qualitative evaluation, we implemented a utility function to export model predictions in NIfTI format for visualization in 3D Slicer. We selected the three best, three average, and three worst test samples based on Dice score. These were visualized using multi-planar views with surface rendering, allowing for anatomic inspection and overlay comparison of predictions (green) and ground truth masks (red). Three examples can be seen in Appendix C.

Sustainability

As part of this course's sustainability focus, we estimated the compute resources used throughout the project and evaluated their environmental impact by converting energy use into a more tangible metric: electric vehicle travel distance.

Total Compute Time

All training jobs were executed on a desktop with an NVIDIA RTX 4090 GPU. We measured training durations using a timer embedded in the training function.

Compute time breakdown:

- **Preselection (9 runs):** 283.54 minutes
- **1st Training Round:** 444.00 minutes
- **2nd Training Round:** 294.00 minutes
- **Estimated overhead (trial/error, sanity checks):** +30%

Total effective compute time:

$(283.54+444+294)\times 1.3 = \mathbf{1{,}345.79\ minutes \approx 22.43\ hours}$

Energy Consumption Estimate

The RTX 4090 has a typical power draw of 450W during training. Thus, the total energy consumption is:

$$\text{Energy (kWh)} = \frac{450 \times 22.43}{1000} = \mathbf{10.09\ kWh}$$

Environmental Impact Equivalent

A Tesla Model Y consumes roughly 0.16 kWh/km. Therefore, with 10.09 kWh, it could travel:

$$\frac{10.09}{0.16} \approx \mathbf{63\ km}$$

This would cover just over half the distance between Trondheim and Oslo, which is approximately 490 km. While this energy footprint is relatively modest for a research project involving deep learning, it highlights the importance of efficient model selection and resource-aware experimentation in computer vision workflows.

Discussion

This project addressed a challenging medical image segmentation task involving 3D MRI volumes for pre-treatment planning in head and neck cancer cases. Despite limited training data and highly imbalanced voxel distributions, our final model achieved strong performance with a test-time Dice score of 0.7024. This result confirms the potential of modern encoder–decoder architectures such as DynUNet, combined with careful preprocessing and patch-based sampling, to effectively segment complex anatomical structures.

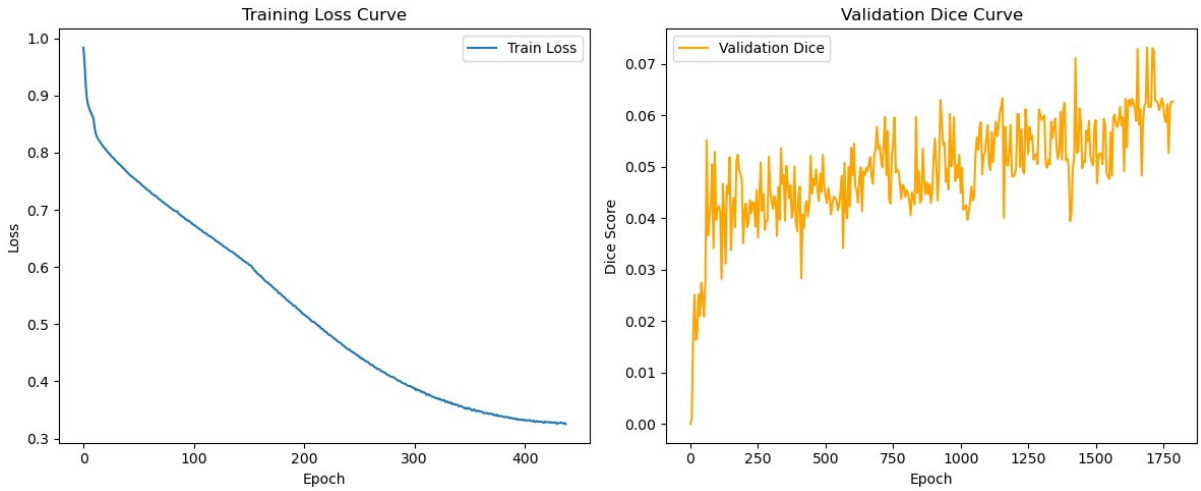

Figure 2: First training round.

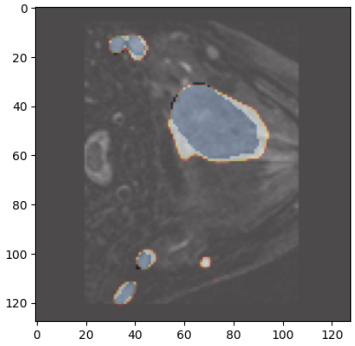

Figure 3: Example from the manual sanity check.

The project also highlighted the sensitivity of medical workflows to implementation details. An initially unnoticed error in the validation Dice calculation led to confusion and re-training. This underlines the need for robust validation pipelines and manual sanity checks when performance metrics do not align with visual or clinical expectations.

From a system design perspective, the modular structure of our codebase was key in facilitating experimentation and debugging. The ability to swap models, losses, and schedulers independently made the preselection process tractable and accelerated the convergence toward a final pipeline.

Room for Improvement and Future Work

There remain several opportunities for improvement:

- **Post-processing:** Morphological operations or CRF-based refinement could help suppress false positives and sharpen boundaries.
- **Ensembling:** Combining predictions from multiple model checkpoints or architectural variants may improve robustness and Dice score stability.
- **Attention-based refinement:** Although Attention UNet underperformed in isolation, integrating attention modules into DynUNet could help localize small or low-contrast tumor regions.
- **Clinical validation:** Incorporating clinical feedback or inter-observer segmentation comparisons would be essential to translate this model into real-world deployment.

Overall, this project demonstrated a complete pipeline from exploratory analysis to deployment-ready predictions, with practical utility in RT planning and room for future optimization.

Key Learning Points

This project was one of the most challenging machine learning tasks I have encountered. Unlike my previous work with tabular datasets or 2D image classification, medical image segmentation on 3D MRI volumes demanded a deeper understanding of computational constraints, spatial context, and model robustness. I learned how to properly structure a complex machine learning project into clear and reusable components, with distinct scripts for data loading, augmentation, model definition, training, evaluation, and visualization. This modular organization was essential for maintaining clarity as the project evolved and new requirements emerged.

A particularly important lesson came from debugging the validation Dice calculation. When

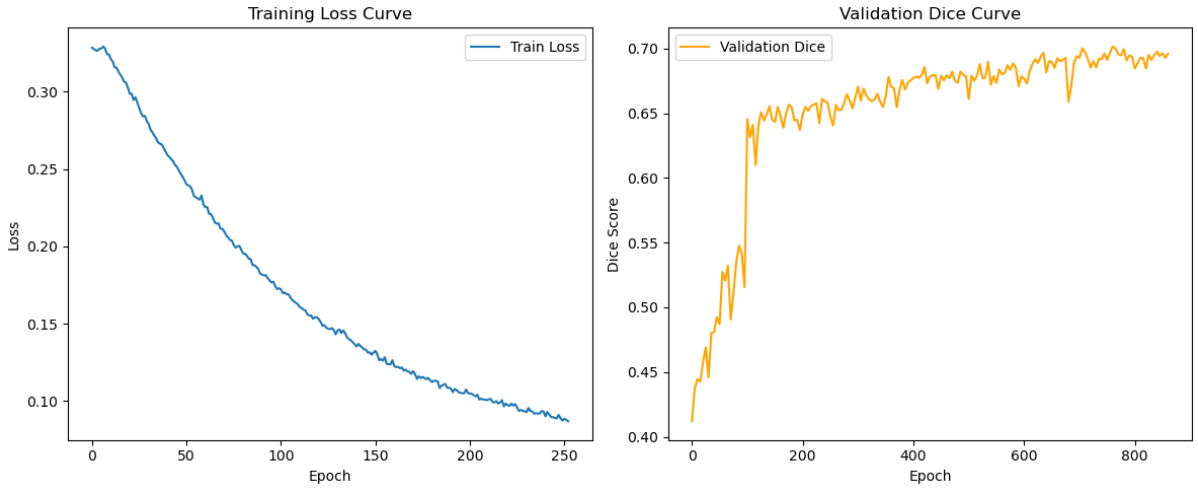

Figure 4: Second training round

metric values did not align with the visual quality of predictions, I realized how critical it is to interpret performance metrics cautiously and to supplement them with manual checks and visual overlays. This experience sharpened my attention to validation logic and the importance of trustworthy evaluation pipelines.

Throughout the project, I also gained hands-on experience with advanced deep learning tools such as MONAI, mixed-precision training, and sliding-window inference—all of which are necessary when working with high-resolution 3D data under limited GPU memory. The structured preselection phase taught me how to run focused, reproducible experiments and compare model configurations meaningfully, rather than relying on intuition alone. Most importantly, working with real clinical MRI data deepened my appreciation for the complexity of biomedical imaging and strengthened my interest in applying deep learning to healthcare. This project has further motivated me to pursue future work in bioinformatics and medical image computing, where I believe machine learning can provide tangible value to both clinicians and patients.

References

[1] J. M. V. Granton, L. Palma, G. Louie, et al., *The role of magnetic resonance imaging in radiation therapy planning*, Seminars in Radiation Oncology, 2014.

[2] M. Hoch and M. Mlynárik, *MRI in radiation therapy planning*, Radiologe, 2006.

[3] B. H. Menze, et al., *The Multimodal Brain Tumor Image Segmentation Benchmark (BRATS)*, IEEE Transactions on Medical Imaging, 2015.

[4] G. Litjens, et al., *A survey on deep learning in medical image analysis*, Medical Image Analysis, 2017.

[5] O. Ronneberger, P. Fischer, and T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, MICCAI, 2015.

[6] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger, *3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation*, MICCAI, 2016.

[7] F. Isensee, P. F. Jäger, S. A. A. Kohl, J. Petersen, and K. H. Maier-Hein, *nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation*, Nature Methods, 2021.

[8] M. J. Cardoso, et al., *MONAI: An open-source framework for deep learning in healthcare imaging*, Nature Methods, 2022.

[9] M. Staff Larsen, *TDT4265: Mini-Project Guidelines*, NTNU, 2025.

[10] I. Loshchilov and F. Hutter, *SGDR: Stochastic Gradient Descent with Warm Restarts*, ICLR, 2017.

[11] A. Fedorov et al., *3D Slicer as an Image Computing Platform for the Quantitative Imaging Network*, Magnetic Resonance Imaging, 2012.

[12] S. S. M. Salehi, D. Erdogmus, and A. Gholipour, *Tversky loss function for image segmentation using 3D fully convolutional deep networks*, MICCAI, 2017.

[13] O. Oktay, J. Schlemper, L. Le Folgoc, M. Lee, M. Heinrich, K. Misawa, K. Mori, S. McDonagh, N. Y. Hammerla, B. Kainz, B.

Glocker, and D. Rueckert, *Attention U-Net: Learning where to look for the pancreas*, arXiv preprint arXiv:1804.03999, 2018.

Appendix

A Extended EDA Visualizations

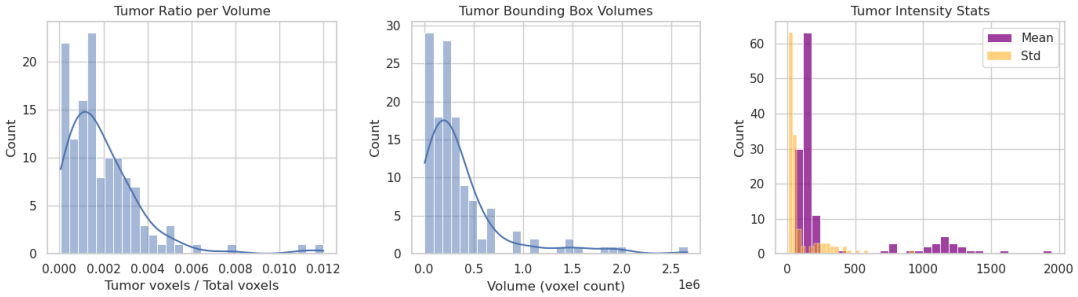

Figure 5: Additional EDA results. Left: Tumor voxel ratio. Center: Bounding box volumes. Right: Tumor region intensity statistics (mean, std).

B Training Loss Curves from Preselection

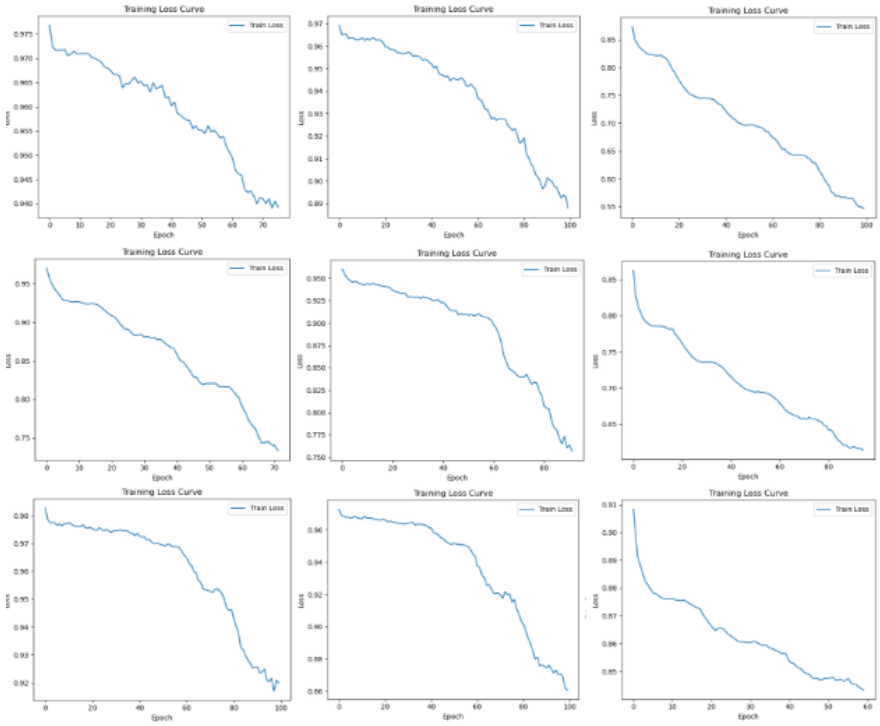

Figure 6: Training loss curves during the model–loss preselection phase. Each plot corresponds to a unique model–loss configuration, ordered left to right, top to bottom as follows:
Top row: (1) UNet + BCE+Dice, (2) UNet + Dice, (3) UNet + Tversky;
Middle row: (4) DynUNet (no DS) + BCE+Dice, (5) DynUNet (no DS) + Dice, (6) DynUNet (no DS) + Tversky;
Bottom row: (7) Attention UNet + BCE+Dice, (8) Attention UNet + Dice, (9) Attention UNet + Tversky.
These curves illustrate the differences in convergence behavior and training dynamics across configurations.

C 3D Slicer Visualizations

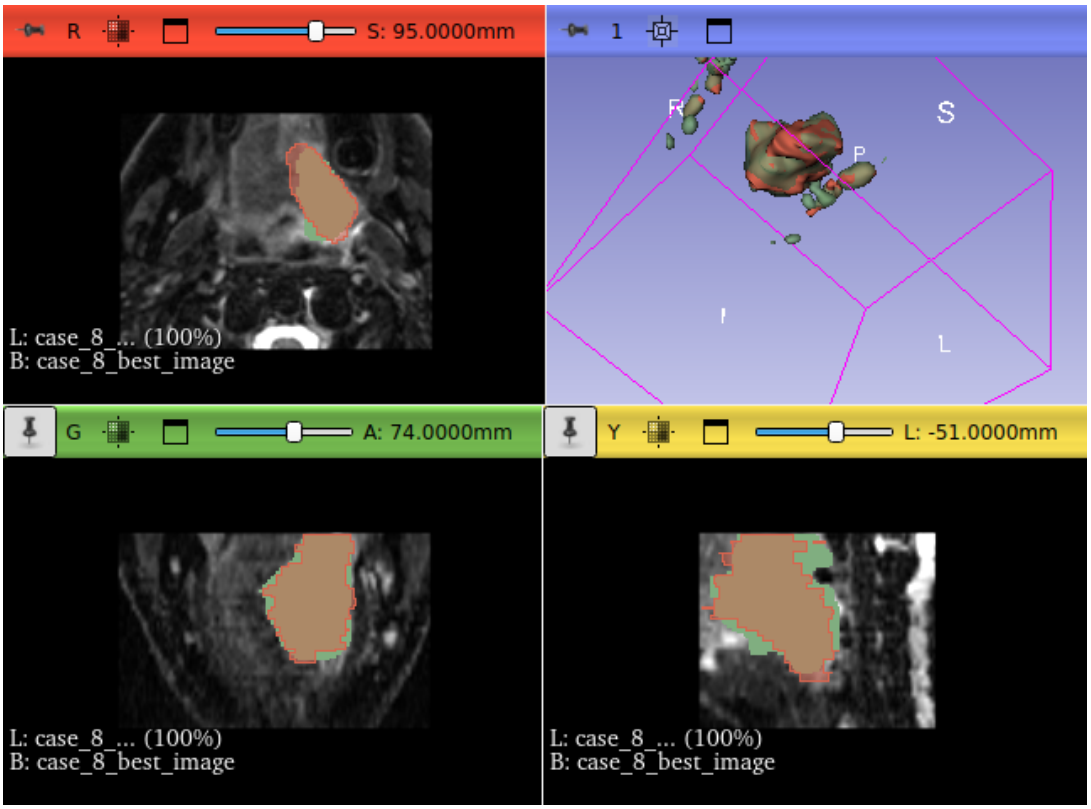

Figure 7: 3D Slicer visualization – Best test case.

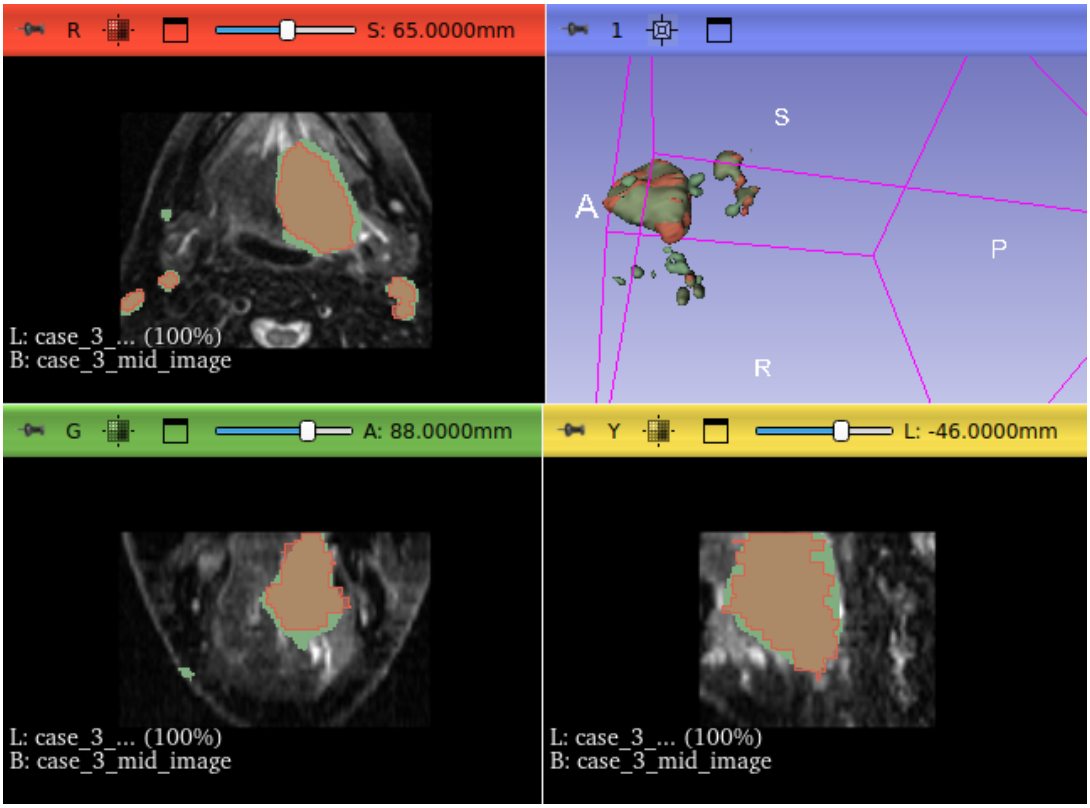

Figure 8: 3D Slicer visualization – Mid-range test case.

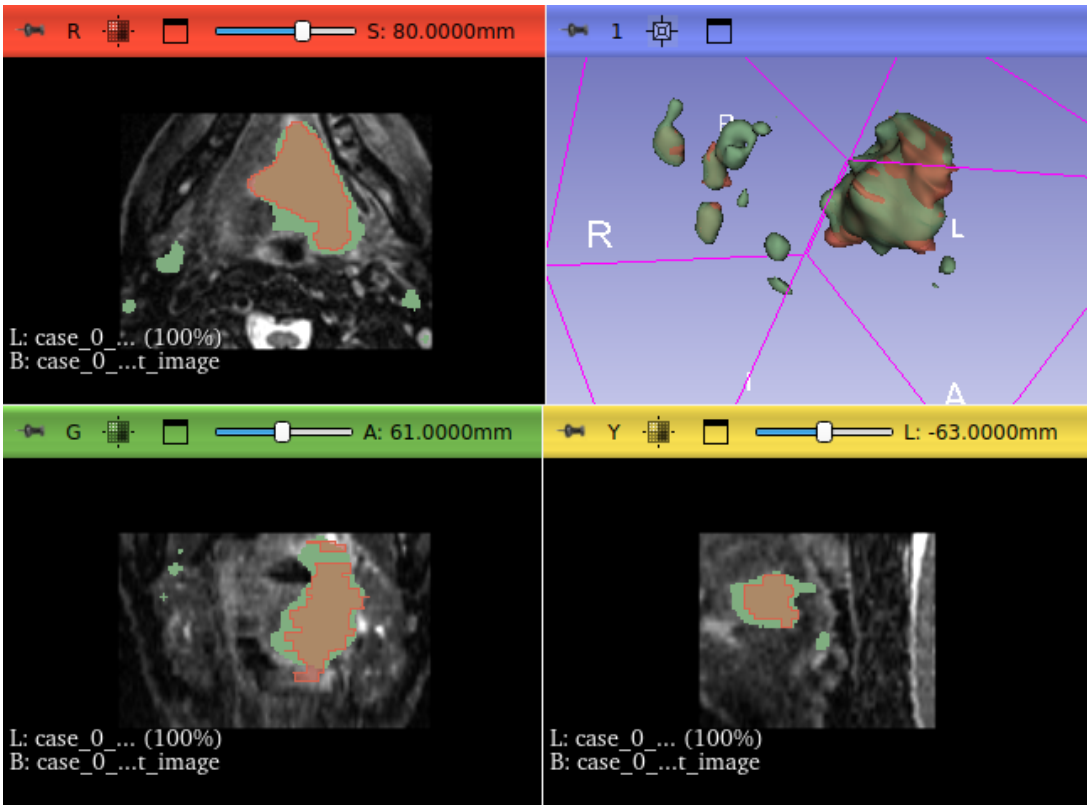

Figure 9: 3D Slicer visualization – One of the worst test cases.